



Request Tracker Agent

Setup & Configuration Guide

A Custom Agent for Microsoft Copilot Studio

Developer: Power CAT | Version 1.0.0.0

Last Updated: May 4, 2026

Table of Contents

1. What Is Request Tracker?
2. Before You Start (Prerequisites)
3. Option A – Install from the Microsoft Marketplace (Agent Library)
4. Option B – Import the Solution into Copilot Studio
5. Option C – Publish to Microsoft Teams & M365 Copilot
6. Option D – Build the Agent from Scratch in Copilot Studio
7. Make It Your Own (Customization)
8. Test That It Works

1. What Is Request Tracker?

The Request Tracker Agent is a Copilot Studio custom agent that helps you manage requests stored in a Microsoft Dataverse table. It provides a conversational interface where you can search, create, update, and analyze requests — all from a chat window in Microsoft Teams or Microsoft 365 Copilot.

What It Does

Capability	What Happens
 Search & Query	Find, filter, and analyze requests by keywords, status, priority, assignee, date, or any combination
 Summarize & Report	Get counts, breakdowns, and trends (e.g., “how many high priority requests are blocked?”)
 Create a Request	Submit a new request via an interactive Adaptive Card form with AI-powered duplicate detection
 Update a Request	Search for an existing request, select it, and edit fields via a pre-populated Adaptive Card form

Built-in Suggested Prompts

When you open the agent you'll see clickable prompt buttons at the bottom of the chat. Just click one to get started — no typing needed:

Button	What It Does
 Search Requests	Show me all open, high priority requests
 Analyze Requests	How many requests have been created this month
 Create Request	Create a request
 Update Request	Update a request

How the Output Is Organized

Search results are displayed in a rich, emoji-enhanced format showing the title, status, priority, requestor, assigned-to, due date, and description for each matching request. Create and Update operations use interactive Adaptive Card forms with dropdown menus and date pickers pre-populated with relevant data. After each action, the agent confirms success with a clear summary.

2. Before You Start

Make sure the items below are in place. If any are missing, contact your IT administrator.

Checklist

✓	What You Need	Why
☐	Microsoft 365 work account	Required to sign in to Copilot Studio and Teams
☐	Microsoft Copilot Studio license	Gives you access to build and run agents in Copilot Studio
☐	Access to a Power Platform environment	The agent and its components live inside an environment
☐	System Administrator role in the target environment	Required to import solutions containing security roles. Request Tracker includes 2 security roles that modify permissions

 **Note:** These are the only universal prerequisites. Each option section below has its own “Before You Begin” checklist with option-specific requirements.

Which Option Should I Choose?

Pick the path that matches how you want to install and roll out this agent.

Option	Best For	You'll Need	Time
Option A – Microsoft Marketplace	Easiest install, ready-to-use experience, no admin needed	Microsoft 365 work account + Power Platform environment + permission to install marketplace apps (System Administrator or System Customizer in target environment)	~10 min
Option B – Import Solution	Fastest setup — uses the pre-built solution package	Copilot Studio access + solution .zip file	~5 min
Option C – Publish to Teams & M365 Copilot	Share the agent with your team via Microsoft Teams	Copilot Studio access + completed Option B	~10 min
Option D – Build from Scratch	Full control — learn how every piece works	Copilot Studio + Power Automate access	~45 min

3. Option A – Install from the Microsoft Marketplace (Agent Library)

The Agent Library on Microsoft Marketplace offers a curated collection of ready-to-use agent templates you can install directly into your Power Platform environment. Browse by category and deploy in minutes. For more information, visit our documentation on [Microsoft Learn](#).

What's Included in the Library

- **Agent templates for real business scenarios.** Each one follows Microsoft best practices and is ready to customize.
- **Two agent types supported.** Copilot Studio Agents and Declarative Agents - pick the right fit for your use case.
- **Guided configuration.** Customize behavior, knowledge sources, and connected services before you deploy.
- **Version tracking.** Know when updates are available and keep your deployed agents current.

Before You Begin

✓	What You Need	Why
☐	Microsoft 365 work account	Required to sign in to Microsoft Marketplace
☐	A target Power Platform environment	The marketplace app installs into a specific environment
☐	Permission to install marketplace apps in that environment	You need System Administrator or System Customizer security role in target environment
☐	Acceptance of the Marketplace terms of service	Required during the "Get it now" flow

How to Install

1. Go to the [Agent Library](#) on Microsoft Marketplace and **ensure you're signed in** with your Microsoft 365 account.

2. Select **Get it now**.
3. Select **Get it now** to confirm.
4. Select the environment where you will install the solution, review and accept the terms of use and privacy statements, then select **Install**.
5. You'll be redirected to Power Platform Admin Center. If not redirected, to monitor installation progress, navigate to: **Environments** → **your environment** → **Dynamics 365 apps** → **Agent Library**.
6. Once installed, go to **make.powerapps.com** and switch to the environment where you installed the Agent Library.
7. Navigate to **Apps** → **Agent Library**.
8. Browse the available templates and deploy your first agent.

 **Tip:** If you don't see the Agent Library or the Get it now button, ask your tenant admin to [enable Marketplace apps](#) for your environment.

 **Important:** This solution enables code components in your environment. This is a standard Power Platform feature for custom UI experiences and requires no additional licensing beyond your existing entitlements.

Deploy Request Tracker from the Library

1. Open the Agent Library app from **make.powerapps.com** → **Apps** → **Agent Library**.
2. Browse or search for **Request Tracker**.
3. Select the template, then select the environment you want to install the agent into
4. Ensure you connect to all of the required data sources (you may need to refresh the connections once they've been configured)
5. Click **Install** (or **Download** if you want to manually import the solution).
6. Once the installation completes, you'll see a "Your agent is ready!" message.
7. Click **Launch in Copilot Studio** to customize and publish.
8. Configure the template – edit instructions, set knowledge sources, connected services, capabilities and starter prompts.
9. Use the Test pane in Copilot Studio to test and verify the agent responds correctly
10. After you've tested, you can publish your agent.

Troubleshooting

Problem	Fix
"Get it now" button is greyed out	Sign in with a work or school account that has Marketplace access (personal Microsoft accounts are not supported)
No environments listed in the install dialog	Confirm you have admin permissions on at least one Power Platform environment
Install stuck in "Installing"	Check Power Platform admin center under Resources → Dynamics 365 apps; allow up to 15 minutes
App isn't opening correctly. " This environment does not allow this operation for this Code app."	Have a Power Platform admin enable Code app operations for the target environment
Agent Library app doesn't appear in make.powerapps.com → Apps	Confirm install completed and that you're viewing the same environment
Request Tracker template not visible in the library	Confirm the agent has been published to the Marketplace catalog

4. Option B – Import the Solution into Copilot Studio

This is the fastest way to get started. You'll upload the pre-built solution package directly into Copilot Studio, which sets up everything — the agent, its topics, AI models, Dataverse table, and security roles — in one step.

Before You Begin

✓	What You Need	Why
☐	Copilot Studio access at copilotstudio.microsoft.com	Where you'll import the solution
☐	System Customizer or System Administrator security role	Required to import solutions
☐	RequestTrackerAgent_1_0_0_0.zip file	The solution package — do not unzip or rename it

 **Important:** Do not unzip, rename, or modify the .zip file. Copilot Studio needs the original package as-is.

Import the Solution

1. Open your browser and go to copilotstudio.microsoft.com. Sign in with your Microsoft 365 work account.
2. Check the environment selector in the top-right corner. Switch to the environment where you want to install the agent if needed.
3. In the left navigation, click Settings (gear icon .
4. Under Settings, click Solutions.
5. Click Import at the top of the Solutions list.
6. Click Browse, select the RequestTrackerAgent_1_0_0_0.zip file, and click Open.
7. Click Next. If prompted for connection references (e.g., Microsoft Dataverse), sign in with your Microsoft 365 account.
8. Click Import and wait for the green success banner.

 **The solution is imported! The agent, topics, AI models, Dataverse table, and security roles are all set up.**

Find and Open Your Agent

1. In the left navigation, click Agents.
2. Find "Request Tracker" in the agent list.

3. Click on it to open the agent overview.

Verify the Dataverse Connection

The agent uses Dataverse connector actions that run in invoker mode — meaning each user's own credentials are used when reading or writing data. After import, verify the connection is active.

1. Open the agent and go to the Topics tab.
2. Open any topic (e.g., "New Request Item").
3. If you see a connection warning banner at the top, click it and sign in with your Microsoft 365 account.
4. The connection should show a green checkmark when authenticated.

 **Tip:** If you don't see a connection warning, the connection is already active. No action needed.

Assign Security Roles

The solution includes two pre-built security roles that control who can do what with request data. After import, assign the appropriate role to each user.

Role	Access Level	Best For
RequestTracker – Admin	Full access to all request records (Organization scope)	Team leads, managers, administrators
RequestTracker – User	Access only to their own requests (User scope)	Individual team members, requestors

1. Open the Power Platform Admin Center at admin.powerplatform.microsoft.com.
2. Select the environment where you imported the solution.
3. Click Settings → Users + permissions → Security roles.
4. Find "RequestTracker – Admin" or "RequestTracker – User".
5. Click the role, then add users or teams to the role.

 **Note:** Users without an assigned security role may not be able to create or view requests. Make sure every agent user has at least the RequestTracker – User role.

Troubleshooting

Problem	Fix
Import button grayed out	You need the System Customizer or System Administrator security role
Import fails with dependency errors	Ensure the environment has Copilot Studio and Dataverse enabled
Agent not appearing after import	Wait 1–2 minutes and refresh the page
Connection errors in topics	Open the topic, click the connection warning, and re-authenticate

Problem	Fix
"Solution already exists" error	Delete the existing solution first or use the overwrite option

5. Option C – Publish to Microsoft Teams & M365 Copilot

After importing the solution (Option B), make the agent available to your colleagues through Microsoft Teams and Microsoft 365 Copilot.

 **Note:** You must complete Option B first.

Step 1: Publish the Agent

1. Open the Request Tracker agent in Copilot Studio.
2. Click Publish in the top-right corner.
3. Wait for the “Published successfully” confirmation.

Step 2: Connect the Microsoft Teams Channel

1. In the agent overview, click Channels in the left navigation.
2. Find Microsoft Teams and click to open it.
3. Turn on the Teams channel toggle.

Configure the Teams App Details

Fill in the following fields to customize how the agent appears in Teams:

1. App Icon — Upload a color icon (192×192 PNG) and an outline icon (32×32 PNG).
2. Accent Color — Enter a hex color code for the agent’s brand color (e.g., #0078D4).
3. Short Description — “A request tracking agent for managing team tasks and requests.”
4. Long Description — “The Request Tracker Agent helps you search, create, update, and analyze requests stored in Dataverse. Use natural language to manage your team’s request workflow.”
5. Developer Information — Enter your organization name, website, privacy policy URL, and terms of use URL.

Enable the Microsoft 365 Copilot Channel

To make the agent available in M365 Copilot as well:

1. Go back to the Channels page.
2. Find Microsoft 365 Copilot and click to open it.
3. Turn on the M365 Copilot channel toggle.
4. Click Save or Apply.

 **Tip:** Enabling the M365 Copilot channel lets users access the agent directly from `m365.cloud.microsoft/chat` — no Teams required.

Step 3: Install for Yourself First

1. Back on the Teams channel page, click “See agent in Teams.”
2. Teams opens with the agent card — click Add.
3. Test the agent by typing a prompt (e.g., “Show me all open requests”).

Step 4: Share with Others

Method 1: Direct Installation Link

1. In Copilot Studio, go to the Teams channel settings.
2. Copy the installation link.
3. Share the link with colleagues via email or Teams chat.

Method 2: Built with Power Platform

1. In Copilot Studio, go to Settings → Availability.
2. Toggle on “Show in Built with Power Platform” for your organization.
3. Users can find the agent in the Teams app store under “Built for your org.”

Method 3: Submit for Admin Approval

1. In Copilot Studio, click Submit for admin approval.
2. Your IT admin reviews and approves the agent in the Teams Admin Center.
3. Once approved, the agent appears in the “Built for your org” section of the Teams app store for all users.

 **Tip:** Method 3 is best for organization-wide rollouts.

How to Access After Publishing

Where	How to Open
Microsoft Teams Chat	Open Teams → click the Copilot icon → find Request Tracker in the agent panel
Teams App Store	Open Teams → Apps → search “Request Tracker” → Add
M365 Copilot Web	Go to m365.cloud.microsoft/chat → open the agent panel → select Request Tracker
@Mention	In any Copilot chat, type @Request Tracker followed by your question

Pin the Agent for Quick Access

1. Find Request Tracker in your Teams sidebar or agent list.
2. Right-click (or click the ⋮ menu) on the agent.
3. Select Pin. The agent will now appear in your left sidebar for instant access.

Troubleshooting

Problem	Fix
"See agent in Teams" not working	You must publish the agent first (Step 1)
Agent not appearing in Teams	Wait 1–2 minutes, then reopen Teams
Users can't find the agent	Check the visibility toggle or admin approval status
Agent returns errors	Verify Dataverse connections are active in the agent's topics
Install link doesn't work on mobile	Search for the agent in the Teams app store instead

6. Option D – Build the Agent from Scratch in Copilot Studio

This option walks you through building the entire agent from the ground up. It's the best choice if you want to deeply understand how every piece works — from the Dataverse table to the topics, AI models, and connector actions.

 **Important:** This is the longest option (~30 min). Use Option B if you just want a working agent quickly.

Before You Begin

✓	What You Need	Why
☐	Copilot Studio access	Where you'll build the agent
☐	Dataverse table access	The agent reads and writes data to a custom Dataverse table
☐	AI Builder access (included with Copilot Studio license)	Used for duplicate detection and record search in topics

This section is organized into 8 parts:

- Part 1: Create the Agent
- Part 2: Configure the Agent Settings & Instructions
- Part 3: Create the Dataverse Table
- Part 4: Add the Dataverse Knowledge Source
- Part 5: Create the "New Request Item" Topic
- Part 6: Create the "Update Existing Request" Topic
- Part 7: Create the "Populate Sample Data" Topic
- Part 8: Assign Security Roles & Test

Part 1: Create the Agent

1. Open copilotstudio.microsoft.com and sign in.
2. In the left navigation, click Agents.
3. Click + New agent.
4. Select Skip to configure to go directly to the agent settings.

Part 2: Configure the Agent Settings & Instructions

Name & Description

Field	What to Enter
Name	Request Tracker

Field	What to Enter
Description	A helpful agent that manages requests stored in a Dataverse table. Supports searching, creating, updating, and analyzing requests.

Instructions

Scroll down to the Instructions box. Copy and paste everything in the box below into that field. This tells the agent how to behave:

```
## Identity & Purpose
You are the Request Tracker Assistant, a helpful and friendly agent that helps users manage requests stored in the _RequestTracker Dataverse table. You support searching, analyzing, creating, and updating request records. Today's date is {Today()}. Use it when answering questions about deadlines, due dates, or anything time-sensitive.

## Tone & Personality
- Be warm, professional, and concise
- Use emojis to make responses visually scannable (📋, ⚡, 🔍, ✅, etc.)
- Always confirm actions before and after performing them
- If something goes wrong, acknowledge the issue clearly and suggest next steps

## Capabilities
You can help users with the following:
- 🔍 Search & Query Requests – Find, filter, and analyze requests by keywords, status, priority, assignee, date, or any combination. You handle all search and query operations conversationally using your _RequestTracker knowledge source.
- 📊 Summarize & Report – Provide counts, summaries, breakdowns, and trends across requests (e.g., “how many high priority requests are blocked?”)
- ➕ Create a Request – Add a new request to the tracker
- ✎ Update a Request – Modify an existing request's fields (Title, Description, Priority, Status, Assigned To, Due Date)

## Search & Query Behavior
You have direct access to the _RequestTracker Dataverse knowledge source. When users ask to search, query, filter, or analyze requests:
- Answer directly from the knowledge source – do not redirect to another topic or ask the user to search separately
- Support natural language queries such as “show me all high priority requests”, “what requests are assigned to John?”, “list blocked requests due this week”
- When filtering, combine multiple criteria naturally (e.g., status + priority + assignee)
- If no results match the query, suggest the user try broader or different keywords
- Always ground search results in the knowledge source data – do not fabricate or guess request records

## Request Fields
- Title – Short name summarizing the request (primary identifier)
- Description – Detailed explanation of what is being requested
- Priority – Urgency level: Low, Medium, High, or Critical
- Status – Lifecycle stage: New, In Progress, Blocked, or Completed
- Assigned To – The person or team responsible for fulfilling the request
- Due Date – Target completion date (format: YYYY-MM-DD)
- Submitted Date – Date the request was originally created
- Requestor – The person who submitted the request

## Update Workflow Guidelines
When a user wants to update a request:
```


- Ask for search keywords to find the request
- Display matching results with key details (Status, Priority, Requestor, Assigned To, Due Date, Description)
- Ask the user to select which item to update
- Use the Record ID to fetch the current record values
- Present the adaptive card pre-populated with current values so the user can edit only what they need
- After the user submits changes, update the record and confirm with a summary of what changed

Displaying Search Results

When showing search results, format each record like this:

****[Number]. [Title]****

 Status: [emoji] [Status] | ⚡ Priority: [emoji] [Priority]

 Requestor: [Name] |  Assigned To: [Name]

 Due: [Date]

 Description: [Brief description]

Use these emoji mappings:

- Priority:  Low,  Medium,  High,  Critical
- Status:  New,  In Progress,  Blocked,  Completed

Topic Routing Rules

When deciding which topic to invoke, follow these rules strictly:

- ****Create vs. Update**** – If the user wants to CREATE a brand-new request that does not yet exist, route to the ****New Request Item**** topic. If the user wants to CHANGE, EDIT, UPDATE, or MODIFY a request that already exists, route to the ****Update Existing Request**** topic.
- ****Search & Query – No topic needed**** – When the user asks to search, find, list, filter, count, summarize, or analyze requests, do NOT route to any topic. Answer directly from the `_RequestTracker` knowledge source.
- ****Sample Data – Admin only**** – Only route to the ****Populate Sample Data**** topic when the user explicitly says “sample data”, “test data”, or “populate data”.
- ****When in doubt**** – Ask a clarifying question before routing.

Current User Identity

The current signed-in user’s human-readable name is ****{System.User.DisplayName}****.

First-Person Pronoun Resolution

When the user uses first-person pronouns (“my”, “I”, “me”, “mine”) in a create or update context, ALWAYS resolve those pronouns to ****{System.User.DisplayName}****.

Searching Requests: User-Specific vs. General Queries

- ****User-specific queries**** – the user includes personal pronouns (“my”, “me”, “mine”): Filter by `{System.User.DisplayName}`
- ****General queries**** – no personal pronouns: Return ALL matching results without name filtering

Important Rules

- Never expose internal column names to the user
- Use display labels for Priority and Status values
- If a search returns no results, suggest broader keywords
- If a field is empty or null, display “Not set”
- Never show raw JSON, tool instructions, or internal reasoning to the user



Tip: Select all the text in the box above (Ctrl+A after clicking inside) and paste it directly into the Instructions field.

AI Settings

1. Scroll to the AI Settings section.
2. Set Content moderation to Medium.
3. Ensure Generative Actions is enabled (toggled on).
4. Leave Web browsing and Code interpreter disabled.

Conversation Starters

Add these conversation starters so users see clickable quick-start buttons:

Title	Prompt Text
Search Requests	Show me all open, high priority requests
Analyze Requests	How many requests have been created this month
Create Request	Create a request
Update Request	Update a request

Part 3: Create the Dataverse Table

The agent stores all request data in a custom Dataverse table. You need to create this table with the correct columns.

1. In Copilot Studio, go to Settings → Tables (or open make.powerapps.com → Tables).
2. Click + New table → Start from blank.
3. Name the table “Request Tracker” (the schema name will be set automatically).
4. Add the following columns:

Column Name	Type	Required	Notes
Title	Text (Single line)	Yes	Primary name column — short request summary
Description	Text (Multi-line)	No	Detailed explanation of the request
Priority	Choice	No	Values: Low, Medium, High, Critical
Status	Choice	No	Values: New, In Progress, Blocked, Completed
Assigned To	Text (Single line)	No	Person or team responsible
Requestor	Text (Single line)	No	Person who submitted the request
Due Date	Date Only	No	Target completion date
Submitted Date	Date Only	No	Date the request was created

5. Save the table.

 **Tip:** The Priority and Status columns are Choice columns. When creating them, add all option values (e.g., Low, Medium, High, Critical for Priority) during column creation.

Part 4: Add the Dataverse Knowledge Source

The agent uses a Dataverse knowledge source to search and query request records directly from the chat. This powers the 🔍 Search & Query capability.

1. Open the Request Tracker agent in Copilot Studio.

2. Go to Knowledge in the left navigation (or the agent's Knowledge tab).
3. Click + Add knowledge.
4. Select Dataverse as the knowledge source type.
5. Search for and select the "Request Tracker" table you created in Part 3.
6. Click Add to enable the knowledge source.

 **Note:** The Dataverse knowledge source allows the agent to answer search and query questions directly from the data — no topic is needed for search operations.

Part 5: Create the "New Request Item" Topic

This topic handles creating new requests. When triggered, it collects request details via an Adaptive Card form, checks for duplicates using an AI Builder model, and creates the record directly in Dataverse using an inline connector action — no separate cloud flow is needed.

Overview of What This Topic Does

Here's the flow of the topic from start to finish:

- The AI extracts a title and description from the user's message (automatic inputs)
- An Adaptive Card form appears with fields for Title, Description, Priority, Status, Assigned To, and Due Date — pre-populated with the extracted values
- The user reviews, edits if needed, and submits the form
- An AI Builder model checks for potential duplicate requests
- If duplicates are found, the user is asked to confirm or cancel
- A Dataverse connector action creates the new row in the Request Tracker table
- The agent confirms the request was created successfully

Step A: Create the Topic

1. In the agent, go to the Topics tab.
2. Click + Add a topic → From blank.
3. Name the topic "New Request Item."
4. Add a trigger description: "Use this topic ONLY when the user wants to CREATE a brand-new request. Example phrases: 'create a new request', 'submit a request', 'add a new request', 'I need to file a request'."

Step B: Configure Automatic Inputs

The topic uses automatic inputs to extract the title and description from the user's message before showing the form.

1. In the topic's input configuration, add two automatic inputs:
2. **TaskTitle** — Description: "Extract a short, descriptive title for the request from the user's message. If the user uses first-person pronouns ('my', 'I', 'me', 'mine'), replace them with the signed-in user's display name."

3. TaskDescription — Description: “Extract a detailed description of the request from the user’s message. Replace any first-person pronouns with the signed-in user’s display name.”

4. Both inputs should have “Should prompt user” set to No (the AI fills them automatically).

Step C: Add the Adaptive Card Form

Add an Adaptive Card Prompt node that displays a form for the user to review and complete the request details.

1. Add an Adaptive Card Prompt node as the first action.
2. Configure the card with these input fields:

Field	Input Type	Options/Notes
Title	Text input	Pre-populated with TaskTitle, required
Description	Text input (multiline)	Pre-populated with TaskDescription, required
 Priority	Dropdown (ChoiceSet)	 Low,  Medium,  High,  Critical (default: Medium)
 Status	Dropdown (ChoiceSet)	 New,  In Progress,  Blocked,  Completed (default: New)
 Assigned To	Text input	Optional — person or team responsible
 Due Date	Date picker	Optional — target completion date

3. Map the card outputs to global variables: Global.title, Global.description, Global.priority, Global.status, Global.assignedto, Global.duedate.

Step D: Add AI Builder Duplicate Detection

After the user submits the form, add an AI Builder model action to check for potential duplicates before creating the record.

1. Add an AI Builder action node (InvokeAIBuilderModelAction) after the Adaptive Card.
2. Set the input: pass Global.title and Global.description concatenated as the search query.
3. Also pass System.User.DisplayName as the user’s name.
4. Store the output in a topic variable (e.g., Topic.DuplicateSearchOutput).
5. The model searches existing requests and returns whether any similar records exist.

Step E: Add Duplicate Check Logic

Add a condition to handle the duplicate detection result.

1. Add a Condition Group node after the AI Builder action.
2. Set the condition: If the duplicate search output is not blank AND does not contain phrases like “no similar requests” or “no matching” — then duplicates were found.
3. In the “duplicates found” branch: Add a Send Activity node showing the potential duplicates, then a Question node asking “Would you still like to create this request?”
4. If the user says Yes → proceed to create the record (Step F).
5. If the user says No → send a message confirming the request was NOT created.

Step F: Add the Dataverse ‘Add a Row’ Connector Action

This is the core action — it creates the request record directly in Dataverse using an inline connector action, not a separate cloud flow.

1. Add a Connector Action node (Microsoft Dataverse → Add a new row).
2. Set the Table name to your Request Tracker table.
3. Map the following fields:

Dataverse Column	Map To
Title (cat_title)	Global.title
Description (cat_description)	Global.description
Priority (cat_priority)	Value(Global.priority)
Status (cat_status)	Value(Global.status)
Assigned To (cat_assignedto)	Global.assignedto
Due Date (cat_duedate)	DateTimeValue(Global.duedate)
Requestor (cat_requestor)	System.User.DisplayName
Submitted Date (cat_submitteddate)	Now()

4. Set the connection mode to Invoker (uses the signed-in user’s credentials).
5. After the connector action, add a Send Activity node: “☒ Thank you for your submission! Your request {Global.title} has been created.”
6. Add the same connector action and confirmation message in the “no duplicates” branch as well (both paths need to create the record).

 **Important:** The Dataverse connector action runs in Invoker mode — each user’s own credentials are used. This ensures security role permissions (Admin vs. User) are enforced per user.

Part 6: Create the “Update Existing Request” Topic

This topic handles updating existing requests. It uses AI Builder to search for matching records, fetches the selected record’s current values from Dataverse, shows a pre-populated Adaptive Card for editing, and writes the changes back to Dataverse.

Overview of What This Topic Does

- The user provides search keywords (or the AI extracts them from the user’s message)
- An AI Builder model searches existing requests and displays matching results
- The user selects which record to update
- A second AI Builder model extracts the Record ID from the selection
- A Dataverse connector action fetches the full record by ID
- Current values are loaded into global variables
- A pre-populated Adaptive Card form appears for the user to edit
- A second Dataverse connector action updates the record with the new values
- The agent confirms the update was successful

Step A: Create the Topic

1. Create a new topic named "Update Existing Request."
2. Add a trigger description: "Use ONLY when the user wants to UPDATE, EDIT, or MODIFY an existing request. Examples: 'update a request', 'edit a request', 'change the status', 'reassign a request'."

Step B: Collect Search Keywords

1. Add a Set Variable node to check if the AI pre-filled any keywords (from the user's original message).
2. Add a Condition Group: if keywords were NOT pre-filled, add a Question node asking "Which request would you like to update? Type the title, keywords, or describe the item you're looking for."
3. Store the keywords in Topic.UpdateItemQuery.

Step C: Search with AI Builder

1. Add an AI Builder action node to search for matching requests.
2. Pass Topic.UpdateItemQuery and System.User.DisplayName as inputs.
3. The model returns a list of matching requests. Display the results with a Send Activity node.

Step D: Select and Identify the Record

1. If keywords were pre-filled, auto-set the selection. Otherwise, add a Question node: "Which item would you like to update? Type the number or title."
2. Add a second AI Builder action that takes the search results and the user's choice, and extracts the Record ID.
3. Add a Condition Group to check if the Record ID was found. If not, send a message: "I wasn't able to identify that request. Please try again." and end the dialog.
4. Store the Record ID in Global.RecordID.

Step E: Fetch Current Values from Dataverse

Use a Dataverse connector action to get the full record so you can pre-populate the edit form.

1. Add a Connector Action node (Microsoft Dataverse → Get a row by ID).
2. Set the Table name to Request Tracker.
3. Set the Record ID to Global.RecordID.text.
4. Set the connection mode to Invoker.
5. After the connector action, add Set Variable nodes to load each field into global variables:
6. Global.title = Topic.GetItemWithOrganization.cat_title
7. Global.description = Topic.GetItemWithOrganization.cat_description
8. Similarly for Global.priority, Global.status, Global.assignedto, and Global.DueDateText (formatted as "yyyy-MM-dd").

Step F: Show the Pre-Populated Edit Form

Add an Adaptive Card Prompt with all fields pre-populated from the current record values.

1. Add an Adaptive Card Prompt node with the same fields as the Create form (Title, Description, Priority, Status, Assigned To, Due Date), but pre-populated with the Global variable values.
2. Map the card outputs to Topic variables (Topic.title, Topic.description, etc.).

Step G: Update the Record in Dataverse

1. Add a Connector Action node (Microsoft Dataverse → Update a row).
2. Set the Table name to Request Tracker.
3. Set the Record ID to Global.RecordID.text.
4. Map the updated fields from the Adaptive Card outputs:

Dataverse Column	Map To
Title (cat_title)	Topic.title
Description (cat_description)	Topic.description
Priority (cat_priority)	Value(Topic.priority)
Status (cat_status)	Value(Topic.status)
Assigned To (cat_assignedto)	Topic.assignedto
Due Date (cat_duedate)	DateTimeValue(Topic.duedate)

5. Add a Send Activity node: ☒ Your request has been updated successfully!"

Part 7: Create the "Populate Sample Data" Topic

This admin-only topic bulk-loads predefined sample request records into the Dataverse table for testing or demo purposes.

Overview of What This Topic Does

- The user asks to "populate sample data" or "load test data"
- The agent asks for confirmation
- If confirmed, a JSON string containing 20 sample records is parsed into a collection
- A Foreach loop iterates over each record
- Inside the loop, a Dataverse connector action creates each record
- After all records are created, the agent confirms success

Step A: Create the Topic

1. Create a new topic named "Populate Sample Data."
2. Add a trigger description: "Use ONLY when the user explicitly asks to populate, load, or generate sample data or test data in bulk."

Step B: Add Confirmation

1. Add a Question node: "Would you like to add sample data to the Request Tracker table in Dataverse?"
2. Use the Boolean entity to capture Yes/No. Add a Condition Group to proceed only if Yes.

Step C: Define the Sample Data

1. Add a Set Variable node with a JSON string containing the sample records. Each record should include: title, description, status (numeric 1–4), priority (numeric 1–4), requestor, assignedto, submitteddate, and duedate.
2. Add a Parse Value node to convert the JSON string into a typed collection (Table of Records).
3. The record schema should define: title (String), description (String), status (Number), priority (Number), requestor (String), assignedto (String), submitteddate (String), duedate (String).

 **Tip:** The sample data uses 'CURRENT_USER' as a placeholder for the requestor and assignedto fields. The connector action formula replaces this with System.User.DisplayName at runtime.

Step D: Loop and Create Records

1. Add a Foreach loop that iterates over the parsed records collection.
2. Inside the loop, add a Dataverse connector action (Add a new row).
3. Map each field from the loop variable:

Dataverse Column	Map To
Title	Topic.LoopValue1.title
Description	Topic.LoopValue1.description
Status	Topic.LoopValue1.status
Priority	Topic.LoopValue1.priority
Requestor	If(Topic.LoopValue1.requestor = "CURRENT_USER", System.User.DisplayName, Topic.LoopValue1.requestor)
Assigned To	If(Topic.LoopValue1.assignedto = "CURRENT_USER", System.User.DisplayName, Topic.LoopValue1.assignedto)
Submitted Date	DateTimeValue(Topic.LoopValue1.submitteddate)
Due Date	DateTimeValue(Topic.LoopValue1.duedate)

4. After the loop, add a Send Activity: "☒ Sample data has been loaded! You now have sample requests in the Request Tracker table."

 **Note:** This topic is for administrators and demo environments only. It creates multiple records at once.

Part 8: Assign Security Roles & Test

Assign Security Roles

If you built the agent from scratch, you'll need to create security roles manually to control data access.

Role	What It Controls
RequestTracker – Admin	Full access to all records (create, read, update, delete across the organization)
RequestTracker – User	Access only to own records (create, read own, update own, delete own)

1. Open make.powerapps.com → Settings → Security roles.
2. Create a new role "RequestTracker – Admin" with Organization-level access to the Request Tracker table.
3. Create a new role "RequestTracker – User" with User-level access to the Request Tracker table.
4. Assign roles to users via the Power Platform Admin Center.

Test the Agent

1. Click Test your agent in the bottom-left corner of Copilot Studio.
2. Try "Show me all open requests" → The agent should search and display results from the knowledge source.
3. Try "Create a request" → The agent should show the Adaptive Card form, check for duplicates, and create the record.
4. Try "Update a request" → The agent should search for records, let you select one, and show the pre-populated edit form.

 **Note:** The first time you test, you may be prompted to authorize the Dataverse connection. Click Allow to proceed.

7. Make It Your Own

If You Imported the Solution (Option B)

1. Open the agent in Copilot Studio.
2. Navigate to Topics, Knowledge, or Settings to make changes.
3. Save and Publish after any edits.

If You Built from Scratch (Option D)

1. Open the agent in Copilot Studio.
2. Navigate to the section you want to change.
3. Save and Publish.

What You Can Change

Customization	How
Change the agent's tone	Edit the Instructions (Part 2)
Add/remove request fields	Edit the Dataverse table columns and update the topic Adaptive Cards
Modify duplicate detection	Update the AI Builder model configuration in the New Request Item topic
Change conversation starters	Edit in agent configuration
Restrict who can create requests	Adjust security roles in Power Platform Admin Center
Add new capabilities	Create additional topics with Dataverse connector actions
Change data source	Point the knowledge source to a different Dataverse table

8. Test That It Works

Quick Smoke Test

#	Try This Prompt	What to Look For
1	Show me all open, high priority requests	Agent returns matching records from Dataverse with emoji formatting
2	Create a request	Agent shows an Adaptive Card form, checks for duplicates, and creates the record
3	Update a request	Agent searches for records, lets you select one, shows pre-populated edit form
4	How many requests have been created this month	Agent analyzes the data and provides a count/summary
5	Populate sample data	Agent confirms and bulk-loads sample test records (admin only)

 **Note:** If the agent doesn't return data for search queries, make sure the Dataverse knowledge source is active and the table has data. Try 'populate sample data' first to load test records.

Troubleshooting

Problem	Fix
Agent doesn't show the Adaptive Card	Check the topic trigger and ensure it routes correctly for create/update
Search returns no results	Verify the Dataverse knowledge source is enabled and the table has records
"Connection not configured" error	Open the topic, click the connector action, and authenticate the Dataverse connection
Updates don't save	Verify the Dataverse connector action is configured with the correct table and field mappings
Sample data doesn't load	Check the Foreach loop and connector action configuration in the Populate Sample Data topic
Duplicate detection not working	Verify the AI Builder model is properly connected in the New Request Item topic

Full Evaluation

For a deeper test, dedicated evaluation files are available alongside the agent package:

File	What It Is
Request Tracker Agent - Evaluation Test Set.csv	10-question test set for Copilot Studio evaluation
Request Tracker Agent - Evaluation Test Plan.pdf	Detailed test plan with pass/fail criteria